

StaySync — Documentación Completa del Proyecto

Sistema de Gestión Hotelera — Arquitectura de Microservicios

Versión: 1.0.0 | Fecha: 2026-06-20

Índice

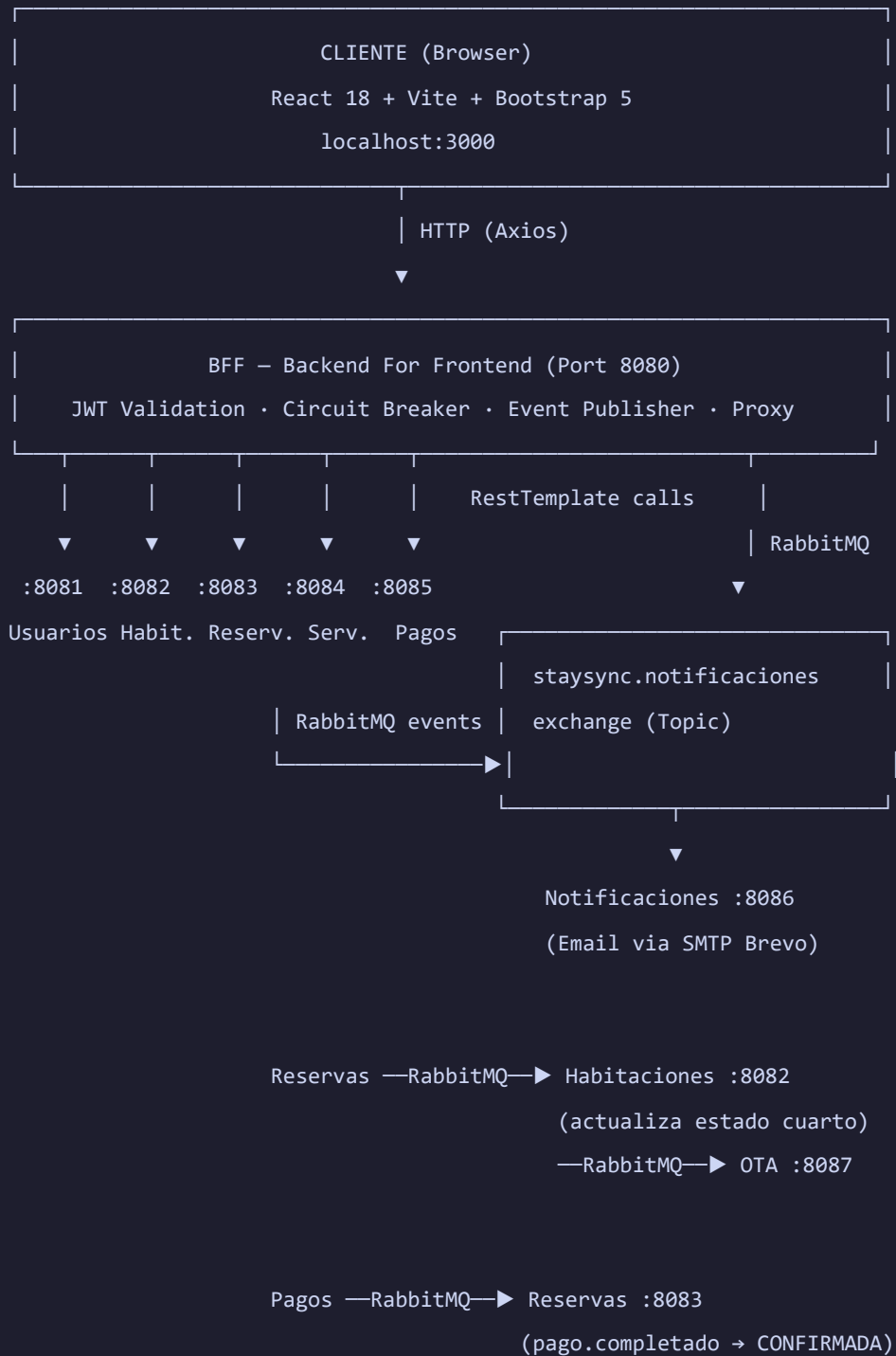
- 1. [Visión General](#)
 - 2. [Arquitectura del Sistema](#)
 - 3. [Stack Tecnológico](#)
 - 4. [Infraestructura y Puertos](#)
 - 5. [Microservicio: BFF \(API Gateway\)](#)
 - 6. [Microservicio: Usuarios](#)
 - 7. [Microservicio: Habitaciones](#)
 - 8. [Microservicio: Reservas](#)
 - 9. [Microservicio: Servicios](#)
 - 10. [Microservicio: Pagos](#)
 - 11. [Microservicio: Notificaciones](#)
 - 12. [Microservicio: OTA](#)
 - 13. [Frontend React](#)
 - 14. [Sistema de Mensajería \(RabbitMQ\)](#)
 - 15. [Seguridad y Autenticación JWT](#)
 - 16. [Resiliencia y Manejo de Fallos](#)
 - 17. [Flujos Completos del Sistema](#)
 - 18. [Variables de Entorno](#)
 - 19. [Cómo Levantar el Proyecto](#)
-

1. Visión General

StaySync es un sistema de gestión hotelera completo que permite administrar:

- **Habitaciones** — inventario, tipos, amenidades, disponibilidad y estados
 - **Reservas** — ciclo de vida completo: PENDIENTE → CONFIRMADA → CHECKIN → CHECKOUT / CANCELADA
 - **Usuarios** — registro, autenticación JWT, roles (ADMIN, RECEPCIONISTA, HUESPED)
 - **Servicios adicionales** — spa, lavandería, restaurante; solicitudes por reserva
 - **Pagos** — procesamiento directo y via Stripe Checkout
 - **Notificaciones** — emails automáticos por eventos (registro, reserva, cambios de estado)
 - **OTA** — integración con canales externos de distribución (Booking.com, Airbnb, etc.)
-

2. Arquitectura del Sistema



Patrón de comunicación

Tipo	Usado para
HTTP sincrónico (RestTemplate)	Consultas en tiempo real (BFF → microservicios, Reservas → Habitaciones)
RabbitMQ asincrónico (Topic exchange)	Eventos: registro usuario, reserva creada, cambio estado, pago completado
BFF pattern	Frontend solo habla con el BFF; el BFF agrega y enruta a los servicios

3. Stack Tecnológico

Backend

Tecnología	Versión	Uso
Java	17	Lenguaje base
Spring Boot	3.4.1	Framework principal
Spring Data JPA	(Boot)	ORM y acceso a datos
Spring Security	(Boot)	Autenticación JWT
Spring AMQP	(Boot)	Mensajería con RabbitMQ
Lombok	(Boot)	Reducción de boilerplate
Resilience4j	2.2.0	Circuit breaker en BFF
MapStruct	1.6.3	Mapeo DTO ↔ Entity
Stripe Java SDK	25.3.0	Pagos con tarjeta
JUnit 5 + Mockito 5	(Boot)	Testing
JaCoCo	0.8.12	Cobertura de código

Base de datos e infraestructura

Tecnología	Versión	Uso
MySQL	8.0	Base de datos (1 por servicio)
RabbitMQ	3.13	Message broker
Docker	-	Contenedores
Nginx	1.27	Reverse proxy del frontend
Maven	3.9.6	Build tool

Frontend

Tecnología	Versión	Uso
React	18	UI framework
Vite	-	Build y dev server
Bootstrap	5	Estilos
Axios	-	HTTP client
React Router	v6	Routing SPA
Vitest	-	Tests unitarios frontend

4. Infraestructura y Puertos

Puertos de servicios

Servicio	Puerto	Base de datos	Puerto MySQL
Frontend	3000 (Nginx)	—	—
BFF	8080	— (sin DB)	—
Usuarios	8081	usuarios_db	interno
Habitaciones	8082	habitaciones_db	interno
Reservas	8083	reservas_db	interno
Servicios	8084	servicios_db	interno
Pagos	8085	pagos_db	interno
Notificaciones	8086	notificaciones_db	interno
OTA	8087	ota_db	interno
RabbitMQ AMQP	5672	—	—
RabbitMQ UI	15672	—	—

Las bases de datos MySQL **no exponen puerto al host** en producción (solo acceso interno entre contenedores Docker).

Healthchecks

Todos los servicios Java exponen:

```
GET /actuator/health → {"status": "UP"}
```

Docker

```
# Levantar todo el stack local (compila desde fuente)
```

```
docker compose up -d
```

Levantar producción (usa imágenes ECR preconstruidas)

```
docker compose -f docker-compose.prod.yml up -d
```

5. Microservicio: BFF (API Gateway) — Puerto 8080

El BFF (Backend For Frontend) es el único punto de entrada para el frontend. Valida JWT, agrega datos de varios servicios y publica eventos a RabbitMQ.

Responsabilidades

1. **Validación JWT** — verifica el token antes de reenviar al microservicio destino
2. **Proxy** — reenvía peticiones a los microservicios con el header `Authorization`
3. **Agregación** — combina datos de varios servicios en una sola respuesta (dashboard)
4. **Circuit Breaker** — si un microservicio falla, retorna fallback en lugar de error
5. **Event Publisher** — publica eventos a RabbitMQ tras acciones críticas

Endpoints

Auth (`/bff/auth`)

Método	Ruta	Descripción	Body
POST	/bff/auth/login	Proxy → usuarios-service login	{email, password}
POST	/bff/auth/registro	Proxy → usuarios-service registro + publica usuario.registro	{nombre, apellido, email, password, rol}
POST	/bff/auth/refresh	Renueva access token	{refreshToken}
POST	/bff/auth/logout	Invalida refresh token	Header: Authorization: Bearer

Habitaciones (/bff/habitaciones)

Método	Ruta	Descripción	Roles
GET	/bff/habitaciones	Lista todas las habitaciones	ADMIN, RECEPCIONISTA
GET	/bff/habitaciones/disponibles	Filtra por capacidad , amenidad , sort=precio_asc\	precio_desc Público
GET	/bff/habitaciones/{id}	Detalle de habitación	Autenticado
POST	/bff/habitaciones	Crear habitación	ADMIN
PATCH	/bff/habitaciones/{id}/estado	Cambiar estado	ADMIN, RECEPCIONISTA

Reservas (/bff/reservas)

Método	Ruta	Descripción	Roles
GET	/bff/reservas	Lista todas con paginación	ADMIN, RECEPCIONISTA
GET	/bff/reservas/hoy	Reservas del día (checkin + checkout pendientes)	ADMIN, RECEPCIONISTA
GET	/bff/reservas/{id}/detalle	Reserva enriquecida con datos de habitación	Autenticado
POST	/bff/reservas	Crear reserva + publica reserva.creada	Autenticado
PATCH	/bff/reservas/{id}/estado	Cambiar estado + publica reserva.estado	ADMIN, RECEPCIONISTA
PATCH	/bff/reservas/{id}/cancelar	Cancelar	Autenticado

Dashboard (/bff/dashboard)

Método	Ruta	Descripción
GET	<code>/bff/dashboard</code>	Agrega stats de reservas y habitaciones para panel de recepción

Respuesta del dashboard:

```
{
  "reservasStats": {
    "total": 42,
    "pendientes": 5,
    "confirmadas": 20,
    "enCheckin": 12,
    "canceladas": 5
  },
  "habitacionesStats": {
    "totalHabitaciones": 30,
    "disponibles": 15,
    "ocupadas": 12,
    "enMantenimiento": 3
  },
  "ultimasReservas": [ ... ]
}
```

Otros endpoints BFF

Prefijo	Descripción
<code>/bff/usuarios</code>	CRUD de usuarios (proxy a usuarios-service)
<code>/bff/servicios</code>	CRUD de servicios y solicitudes (proxy a servicios-service)
<code>/bff/pagos</code>	Procesamiento de pagos + Stripe (proxy a pagos-service)

Cientes HTTP (RestTemplate con Circuit Breaker)

```
UsuariosClient    → http://localhost:8081
HabitacionesClient → http://localhost:8082
ReservasClient   → http://localhost:8083
ServiciosClient  → http://localhost:8084
PagosClient      → http://localhost:8085
```

Eventos RabbitMQ publicados

Routing Key	Evento	Cuándo
usuario.registro	UsuarioRegistradoEvent	POST /bff/auth/registro exitoso
reserva.creada	ReservaCreadaEvent	POST /bff/reservas exitoso
reserva.estado	ReservaEstadoCambiadoEvent	PATCH /bff/reservas/{id}/estado exitoso

6. Microservicio: Usuarios — Puerto 8081

Gestión completa de cuentas de usuario, autenticación JWT y refresh tokens.

Modelos de datos

Usuario (tabla: **usuarios**)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
nombre	VARCHAR(100)	Nombre
apellido	VARCHAR(100)	Apellido
email	VARCHAR(150) UNIQUE	Email (login)
passwordHash	VARCHAR(255)	BCrypt hash
telefono	VARCHAR(20)	Opcional
rol	ENUM	ADMIN , RECEPCIONISTA , HUESPED
activo	BOOLEAN	Soft delete
createdAt / updatedAt	DATETIME	Auditoría

RefreshToken (tabla: refresh_tokens)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
usuario_id	BIGINT FK	→ usuarios.id
token	VARCHAR(255) UNIQUE	UUID generado
expiraEn	DATETIME	7 días desde creación
createdAt	DATETIME	Auditoría

Endpoints

Auth (/api/v1/auth) — Público

Método	Ruta	Body	Respuesta
POST	/api/v1/auth/registro	RegistroRequest	UsuarioResponse (201)
POST	/api/v1/auth/login	LoginRequest	AuthResponse (200)
POST	/api/v1/auth/refresh	{refreshToken}	AuthResponse (200)
POST	/api/v1/auth/logout	{refreshToken}	(204)

RegistroRequest :

```
{
  "nombre": "Juan",
  "apellido": "García",
  "email": "juan@hotel.com",
  "password": "MiPass@123",
  "rol": "HUESPED"
}
```

AuthResponse :

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "refreshToken": "550e8400-e29b-41d4-a716-...",
  "email": "juan@hotel.com",
  "rol": "HUESPED",
  "expiresIn": 86400000
}
```

Usuarios (`/api/v1/usuarios`) — Protegido

Método	Ruta	Descripción	Rol mínimo
GET	<code>/api/v1/usuarios</code>	Lista paginada	ADMIN
GET	<code>/api/v1/usuarios/huespedes</code>	Solo huéspedes	RECEPCIONISTA
GET	<code>/api/v1/usuarios/buscar?q=...&rol=...</code>	Búsqueda	RECEPCIONISTA
GET	<code>/api/v1/usuarios/{id}</code>	Por ID	RECEPCIONISTA
GET	<code>/api/v1/usuarios/perfil</code>	Perfil propio	HUESPED
PUT	<code>/api/v1/usuarios/{id}</code>	Actualizar cualquier usuario	ADMIN
PUT	<code>/api/v1/usuarios/perfil</code>	Actualizar perfil propio	HUESPED
DELETE	<code>/api/v1/usuarios/{id}</code>	Desactivar (soft delete)	ADMIN

Seguridad

- Contraseñas hasheadas con **BCrypt**
 - Access token válido **24 horas** (`${jwt.expiration-ms}`)
 - Refresh token válido **7 días** (almacenado en BD)
 - Endpoints `/api/v1/auth/` **son** públicos**; todo lo demás requiere JWT
-

7. Microservicio: Habitaciones — Puerto 8082

Inventario completo de habitaciones del hotel con tipos, amenidades y estados.

Modelos de datos

`Habitacion` (tabla: `habitaciones`)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
numero	VARCHAR(10) UNIQUE	Número de habitación (ej: "101", "302A")
piso	INTEGER	Piso del edificio
tipo_id	BIGINT FK	→ tipos_habitacion.id
estado	ENUM	DISPONIBLE , OCUPADA , EN_LIMPIEZA , MANTENIMIENTO , FUERA_DE_SERVICIO
descripcion	TEXT	Descripción detallada
precioPorNoche	DECIMAL(10,2)	Precio base por noche
activa	BOOLEAN	Soft delete
amenidades	Many-to-Many	→ amenities (tabla: habitacion_amenidades)
imagenes	One-to-Many	→ imagenes_habitacion
createdAt / updatedAt	DATETIME	Auditoría

TipoHabitacion (tabla: tipos_habitacion)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
nombre	VARCHAR(100) UNIQUE	ej: "Suite", "Doble", "Individual"
descripcion	TEXT	Descripción
capacidad	INTEGER	Máximo de huéspedes
precioBase	DECIMAL(10,2)	Precio base del tipo
activo	BOOLEAN	Disponible para asignar

Amenidad (tabla: amenities)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
nombre	VARCHAR(100) UNIQUE	ej: "WiFi", "TV", "Jacuzzi"
descripcion	TEXT	Descripción
icono	VARCHAR(50)	Clase de icono Bootstrap/Font Awesome

Endpoints (/api/v1/habitaciones)

Método	Ruta	Descripción	Rol
GET	/	Lista todas las activas	RECEPCIONISTA
GET	/disponibles? capacidad=2&amenidad=WiFi&sort=precio_asc	Filtra disponibles	Público
GET	/id	Detalle por ID	Autenticado
GET	/numero/{numero}	Detalle por número	Autenticado
POST	/	Crear habitación	ADMIN
PATCH	/id/estado	Cambiar estado	ADMIN, RECEPCIONISTA
DELETE	/id	Desactivar (soft delete)	ADMIN

Parámetros de buscarDisponibles:

- capacidad — mínimo de personas requerido (filtra por tipoHabitacion.capacidad >= valor)
- amenidad — nombre de amenidad requerida (ej: "WiFi")
- sort — precio_asc , precio_desc , numero_asc (default)

Eventos RabbitMQ

Escucha (Listener)

Queue	Routing Key	Acción
q.habitacion.estado	reserva.checkin	Cambia estado de habitación a OCUPADA
q.habitacion.estado	reserva.checkout	Cambia estado de habitación a DISPONIBLE

Publica (Publisher)

Exchange	Routing Key	Payload
staysync.habitaciones.exchange	habitacion.estado.cambiado	{habitacionId, estadoAnterior, estadoNuevo, timestamp}
staysync.habitaciones.exchange	habitacion.disponibilidad.actualizada	{habitacionId, fecha, disponible, precio}

8. Microservicio: Reservas — Puerto 8083

Gestión del ciclo de vida completo de una reserva hotelera.

Modelos de datos

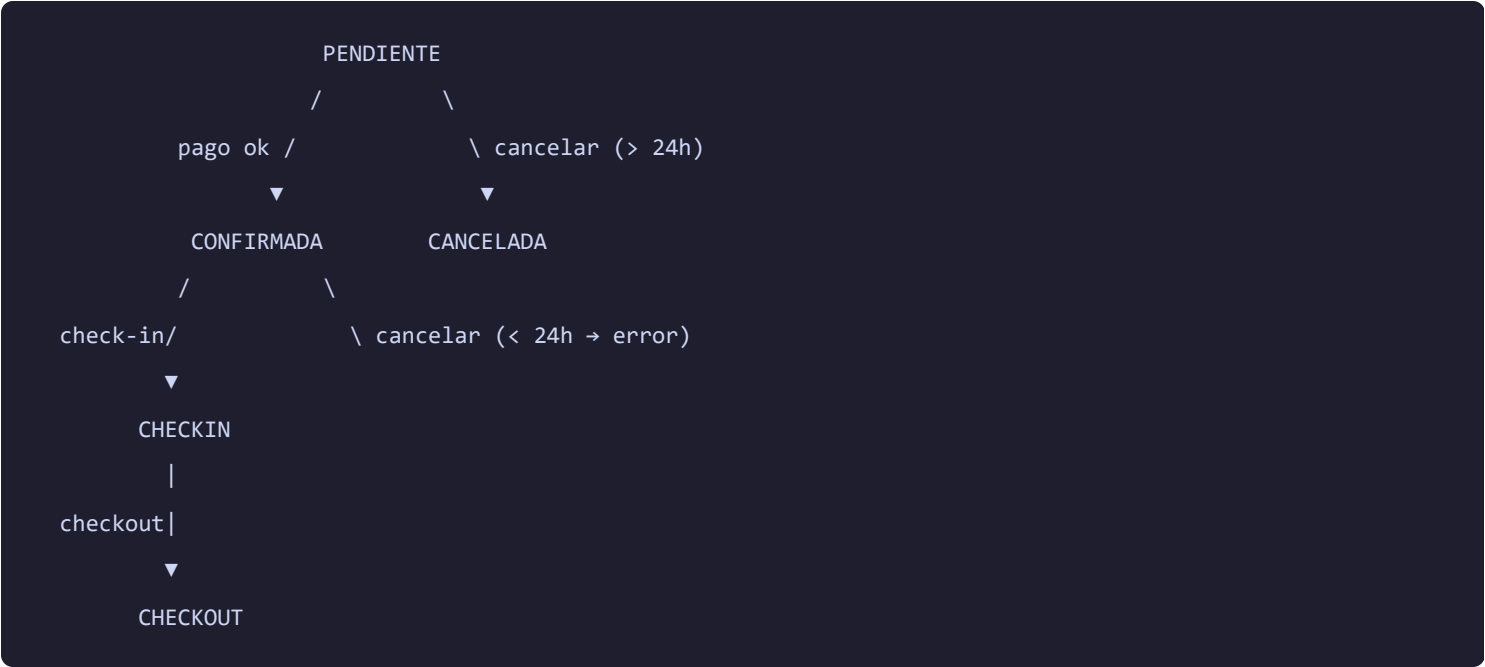
Reserva (tabla: reservas)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
codigo	VARCHAR(20) UNIQUE	Generado: <code>RES-XXXXXXX</code> (8 chars alfanuméricos)
usuario_id	BIGINT	Usuario propietario de la reserva
habitacion_id	BIGINT	Habitación reservada
fechaEntrada	DATE	Día de check-in
fechaSalida	DATE	Día de check-out
numHuespedes	INTEGER	Total de personas
estado	ENUM	<code>PENDIENTE</code> , <code>CONFIRMADA</code> , <code>CHECKIN</code> , <code>CHECKOUT</code> , <code>CANCELADA</code> , <code>NO_SHOW</code>
fuelle	ENUM	<code>DIRECTO</code> , <code>OTA</code> , <code>TELEFONO</code> , <code>WEB</code>
precioTotal	DECIMAL(10,2)	Calculado: <code>precioPorNoche × noches</code>
notas	TEXT	Notas internas
createdAt / updatedAt	DATETIME	Auditoría

`HuespedAdicional` (tabla: `huespedes_adicionales`)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
reserva_id	BIGINT FK	→ reservas.id
nombre	VARCHAR(100)	Nombre
apellido	VARCHAR(100)	Apellido
documento	VARCHAR(50)	DNI / Pasaporte

Máquina de estados de una reserva



Regla de cancelación: Solo se puede cancelar con **más de 24 horas** de anticipación al check-in. Intentar cancelar con menos de 24h lanza `CancelacionRestringidaException` (HTTP 422).

Transiciones inválidas lanzan `TransicionEstadoInvalidaException` (HTTP 400). Ejemplo: CHECKOUT → CHECKIN.

Endpoints (`/api/v1/reservas`)

Método	Ruta	Descripción	Rol
POST	<code>/</code>	Crear reserva	Autenticado
GET	<code>/</code>	Lista paginada	RECEPCIONISTA
GET	<code>/hoy</code>	Reservas del día actual	RECEPCIONISTA
GET	<code>/id</code>	Por ID	Autenticado
GET	<code>/codigo/codigo</code>	Por código	Autenticado
GET	<code>/usuario/usuarioId</code>	Por usuario (paginado)	Autenticado
GET	<code>/habitacion/habitacionId/conflictos</code>	Verificar disponibilidad	Autenticado
PATCH	<code>/id/estado</code>	Cambiar estado	RECEPCIONISTA
PATCH	<code>/id/cancelar</code>	Cancelar reserva	Autenticado

Body de `CrearReservaRequest` :

```
{
  "usuarioId": 1,
  "habitacionId": 5,
  "fechaEntrada": "2026-12-01",
  "fechaSalida": "2026-12-05",
  "numHuespedes": 2,
  "fuente": "DIRECTO",
  "notas": "Solicitan cuna para bebé",
  "huespedesAdicionales": [
    { "nombre": "María", "apellido": "López", "documento": "12345678" }
  ]
}
```

Respuesta `GET /hoy` :

```
{
  "fecha": "2026-12-01",
  "pendientesCheckin": [
    { ...ReservaResponse con estado CONFIRMADA y fechaEntrada = hoy... }
  ],
  "pendientesCheckout": [
    { ...ReservaResponse con estado CHECKIN... }
  ]
}
```

Scheduler (tareas automáticas)

`autoTransicionarACheckin()` — se ejecuta automáticamente (cron) y cambia a `CHECKIN` todas las reservas `CONFIRMADA` cuya `fechaEntrada` sea hoy.

Llamadas HTTP externas

```
GET http://habitaciones:8082/api/v1/habitaciones/{id}
```

→ obtener precioPorNoche y capacidad para validación y cálculo de precioTotal

Eventos RabbitMQ

Escucha (Listener)

Queue	Routing Key	Acción
q.pago.completado	pago.completado	Cambia reserva a CONFIRMADA
q.pago.fallido	pago.fallido	Log warning (reserva permanece PENDIENTE)

Publica (Publisher)

Exchange: `staysync.reservas.exchange`

Routing Key	Cuándo	Consumidor
reserva.confirmada	Estado → CONFIRMADA	Notificaciones
reserva.checkin	Estado → CHECKIN	Habitaciones (→ OCUPADA)
reserva.checkout	Estado → CHECKOUT	Habitaciones (→ DISPONIBLE)
reserva.cancelada	Cancelada	Notificaciones

9. Microservicio: Servicios — Puerto 8084

Catálogo de servicios adicionales del hotel (spa, lavandería, etc.) y gestión de solicitudes.

Modelos de datos

Servicio (tabla: **servicios**)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
nombre	VARCHAR(100)	ej: "Masaje 60min", "Lavandería Express"
descripcion	TEXT	Detalle del servicio
precio	DECIMAL(10,2)	Precio unitario
categoria_id	BIGINT FK	→ categorias_servicio.id
disponible	BOOLEAN	Si está activo actualmente
requiereReserva	BOOLEAN	Si necesita estar hospedado
createdAt / updatedAt	DATETIME	Auditoría

SolicitudServicio (tabla: **solicitudes_servicio**)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
reserva_id	BIGINT	Reserva asociada
usuario_id	BIGINT	Usuario que solicita
servicio_id	BIGINT FK	→ servicios.id
cantidad	INTEGER	Unidades solicitadas
fechaServicio	DATE	Fecha deseada
estado	ENUM	PENDIENTE , EN_PROCESO , COMPLETADO , CANCELADO
precioTotal	DECIMAL	cantidad × servicio.precio
notas	TEXT	Instrucciones especiales

Endpoints (**/api/v1**)

Método	Ruta	Descripción	Rol
GET	/servicios	Lista servicios disponibles	Público
GET	/servicios/{id}	Detalle de servicio	Autenticado
GET	/solicitudes	Todas las solicitudes	RECEPCIONISTA
GET	/solicitudes/reserva/{reservaId}	Por reserva	Autenticado
GET	/solicitudes/usuario/{usuarioId}	Por usuario	Autenticado
POST	/solicitudes	Crear solicitud	HUESPED
PATCH	/solicitudes/{id}/estado	Actualizar estado	RECEPCIONISTA

10. Microservicio: Pagos — Puerto 8085

Procesamiento de pagos con soporte para pagos directos y checkout via Stripe.

Modelo de datos

Pago (tabla: pagos)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
referencia	VARCHAR(50) UNIQUE	Generada: <code>PAY-XXXXXXXX</code> o <code>STRIPE-XXXXXXXX</code>
reserva_id	BIGINT	Reserva asociada
usuario_id	BIGINT	Usuario que paga
monto	DECIMAL(10,2)	Monto pagado
moneda	VARCHAR(3)	<code>USD</code> , <code>CLP</code> , etc.
metodoPago	ENUM	<code>TARJETA_CREDITO</code> , <code>TARJETA_DEBITO</code> , <code>TRANSFERENCIA</code> , <code>EFFECTIVO</code> , <code>PAYPAL</code> , <code>MERCADO_PAGO</code> , <code>STRIPE</code>
estado	ENUM	<code>PENDIENTE</code> , <code>PROCESANDO</code> , <code>COMPLETADO</code> , <code>FALLIDO</code> , <code>REEMBOLSADO</code> , <code>PARCIALMENTE_REEMBOLSADO</code>
gatewayId	VARCHAR(255)	ID en Stripe (ej: <code>cs_test_ABC123</code>)
gatewayRespuesta	JSON	Respuesta completa del gateway
descripcion	TEXT	Descripción legible
createdAt / updatedAt	DATETIME	Auditoría

Endpoints de pagos directos (`/api/v1/pagos`)

Método	Ruta	Descripción
POST	<code>/</code>	Procesar pago directo
GET	<code>/ {id}</code>	Por ID
GET	<code>/referencia/{referencia}</code>	Por referencia
GET	<code>/reserva/{reservaId}</code>	Lista por reserva
POST	<code>/ {pagoId}/reembolso</code>	Solicitar reembolso

Endpoints de Stripe (`/api/pagos/stripe`)

Método	Ruta	Descripción
POST	<code>/checkout</code>	Crear sesión de pago en Stripe → retorna <code>checkoutUrl</code>
POST	<code>/confirmar</code>	Confirmar pago tras redirect de Stripe

Flujo Stripe Checkout:

1. Frontend llama `POST /api/pagos/stripe/checkout` → recibe `{checkoutUrl, sessionId}`
2. Frontend redirige al usuario a `checkoutUrl` (página de pago de Stripe)
3. Stripe redirige al usuario a `successUrl` o `cancelUrl`
4. Frontend llama `POST /api/pagos/stripe/confirmar` con `{sessionId}` → registra el pago

Idempotencia: Si se llama a `/confirmar` con el mismo `sessionId` dos veces, retorna el pago existente sin crear duplicado (`existsByGatewayId`).

Arquitectura de Stripe

```

StripeService
├── StripeGateway (interface)
│   ├── createSession(params) → Session
│   └── retrieveSession(sessionId) → Session
└── StripeGatewayImpl (implementación real que llama al SDK)
    ├── Session.create(params) ← Stripe SDK static call
    └── Session.retrieve(id) ← Stripe SDK static call
  
```

`StripeGateway` es una interfaz que separa la lógica de negocio de las llamadas estáticas del SDK, permitiendo tests sin `MockedStatic`.

Eventos RabbitMQ publicados

Exchange	Routing Key	Cuándo
<code>staysync.pagos.exchange</code>	<code>pago.completado</code>	Pago procesado exitosamente
<code>staysync.pagos.exchange</code>	<code>pago.fallido</code>	Pago rechazado o sin token

11. Microservicio: Notificaciones — Puerto 8086

Servicio 100% event-driven. Escucha eventos de RabbitMQ y envía emails automáticos.

Modelos de datos

Notificacion (tabla: **notificaciones**)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
plantilla_id	BIGINT FK	→ plantillas_notificacion.id
usuario_id	BIGINT	Usuario destinatario
canal	ENUM	EMAIL , SMS , PUSH
destinatario	VARCHAR(255)	Email / teléfono
asunto	VARCHAR(255)	Asunto del email
cuerpo	LONGTEXT	HTML renderizado con variables
estado	ENUM	PENDIENTE , ENVIADO , FALLIDO , CANCELADO
intentos	INTEGER	Contador de intentos (máx 3)
maxIntentos	INTEGER	Default: 3
errorMsg	TEXT	Mensaje del error SMTP
enviadoEn	DATETIME	Timestamp de envío exitoso
createdAt / updatedAt	DATETIME	Auditoría

PlantillaNotificacion (tabla: **plantillas_notificacion**)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
codigo	VARCHAR(50) UNIQUE	Identificador: <code>USUARIO_REGISTRO</code> , <code>RESERVA_CREADA</code> , <code>RESERVA_ESTADO</code>
nombre	VARCHAR(100)	Nombre descriptivo
canal	ENUM	Canal para esta plantilla
asunto	VARCHAR(255)	Asunto con variables: <code>Reserva {{codigo}} confirmada</code>
cuerpo	LONGTEXT	HTML con variables: <code>{{nombre}}</code> , <code>{{codigo}}</code> , etc.
activa	BOOLEAN	Si está activa

Eventos RabbitMQ escuchados

Queue	Routing Key	Evento	Email enviado
<code>q.notificacion.usuario.registro</code>	<code>usuario.registro</code>	<code>UsuarioRegistradoEvent</code>	Bienvenida al nuevo usuario
<code>q.notificacion.reserva.creada</code>	<code>reserva.creada</code>	<code>ReservaCreadaEvent</code>	Confirmación de reserva
<code>q.notificacion.reserva.estado</code>	<code>reserva.estado</code>	<code>ReservaEstadoCambiadoEvent</code>	Notificación de cambio de estado

Eventos (Java Records)

```
public record UsuarioRegistradoEvent(Long usuarioId, String nombre, String email) {}
```

```
public record ReservaCreadaEvent(
    Long reservaId, Long usuarioId, String email, String nombreUsuario,
    String codigo, String habitacion, String fechaEntrada, String fechaSalida,
    Double precioTotal) {}
```

```
public record ReservaEstadoCambiadoEvent(
    Long reservaId, Long usuarioId, String email, String nombreUsuario,
    String codigo, String habitacion, String estadoNuevo) {}
```

Flujo de envío de email

```
1. RabbitMQ listener recibe evento

1. Busca PlantillaNotificacion por código (USUARIO_REGISTRO, etc.)
2. Renderiza plantilla: reemplaza {{variable}} con valores del evento
3. Crea registro Notificacion (estado: PENDIENTE)
4. Llama enviarEmail()

  └─ Éxito: estado = ENVIADO, enviadoEn = now
  └─ Fallo: intentos++

        └─ intentos < maxIntentos: estado = PENDIENTE (reintento)

        └─ intentos >= maxIntentos: estado = FALLIDO, errorMsg guardado
```

Configuración de email

```
# Brevo (producción)
spring.mail.host: smtp-relay.brevo.com
spring.mail.port: 587
```

Flag para activar/desactivar envío real

```
notificaciones.mail.enabled: true # false = solo log, no envía
```

Dead Letter Queue (DLQ)

Si el procesamiento del mensaje falla 3 veces, RabbitMQ mueve el mensaje a la DLQ correspondiente para análisis manual:

- `q.notificacion.usuario.registro.dlq`
- `q.notificacion.reserva.creada.dlq`
- `q.notificacion.reserva.estado.dlq`

12. Microservicio: OTA — Puerto 8087

Integración con canales externos de distribución (Online Travel Agencies).

Modelos de datos

`CanalOta` (tabla: `canales_ota`)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
nombre	VARCHAR(100) UNIQUE	ej: "Booking.com", "Airbnb"
codigo	VARCHAR(20) UNIQUE	ej: <code>BOOKING</code> , <code>AIRBNB</code>
apiUrl	VARCHAR(255)	URL de la API del canal
activo	BOOLEAN	Si el canal está activo
comision	DECIMAL(5,2)	% de comisión del canal

`ReservaOta` (tabla: `reservas_ota`)

Campo	Tipo	Descripción
id	BIGINT PK	Auto-incremento
canal_id	BIGINT FK	→ canales_ota.id
reserva_id_local	BIGINT	ID en StaySync
reserva_id_ext	VARCHAR(100) UNIQUE	ID en el canal externo
estado_ext	VARCHAR(50)	Estado en el sistema externo
payloadEntrada	JSON	Payload recibido del canal
payloadSalida	JSON	Payload enviado al canal
sincronizada	BOOLEAN	Si ya fue procesada
errorMsg	TEXT	Error de sincronización

Endpoints (/api/v1)

Método	Ruta	Descripción
GET	/canales	Lista canales OTA activos
GET	/canales/{id}	Detalle de canal
GET	/sincronizacion/pendientes	Reservas OTA pendientes de sincronizar
POST	/webhooks/{codigoCanal}	Webhook receptor de reservas externas

Eventos RabbitMQ escuchados

Queue	Acción
q.habitacion.estado	Actualiza disponibilidad en canales OTA cuando cambia estado de habitación

13. Frontend React — Puerto 3000

Tech stack

- **React 18** — componentes funcionales con hooks
- **Vite** — dev server rápido, build optimizado
- **Bootstrap 5** — estilos responsive
- **Axios** — HTTP client con interceptors para JWT
- **React Router v6** — navegación SPA

Variables de entorno (`.env`)

```
VITE_BFF_URL=http://localhost:8080/bff
VITE_PAGOS_URL=http://localhost:8085
VITE_MP_PUBLIC_KEY=TEST-xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx # Mercado Pago (si aplica)
```

Scripts

```
npm run dev      # Servidor de desarrollo (hot reload)
npm run build    # Build de producción → /dist
npm run test     # Tests unitarios con Vitest
npm run coverage # Reporte de cobertura
```

Build y producción

El build de producción (`/dist`) se sirve via **Nginx 1.27** con:

- Compresión gzip para JS/CSS/imágenes
 - Cache HTTP de 1 año para assets estáticos
 - SPA fallback: cualquier ruta desconocida → `index.html`
 - Health check en `/health`
-

14. Sistema de Mensajería (RabbitMQ)

Exchanges y bindings

staysync.notificaciones (Topic Exchange)

- |— usuario.registro → q.notificacion.usuario.registro
- |— reserva.creada → q.notificacion.reserva.creada
- |— reserva.estado → q.notificacion.reserva.estado

DLX: staysync.notificaciones.dlx (Direct)

- |— usuario.registro → q.notificacion.usuario.registro.dlq
- |— reserva.creada → q.notificacion.reserva.creada.dlq
- |— reserva.estado → q.notificacion.reserva.estado.dlq

staysync.reservas.exchange (Topic Exchange)

- |— reserva.checkin → q.habitacion.estado (Habitaciones)
- |— reserva.checkout → q.habitacion.estado (Habitaciones)
- |— reserva.* → q.ota.sincronizacion (OTA)

staysync.pagos.exchange (Topic Exchange)

- |— pago.completado → q.pago.completado (Reservas)
- |— pago.fallido → q.pago.fallido (Reservas)

staysync.habitaciones.exchange (Topic Exchange)

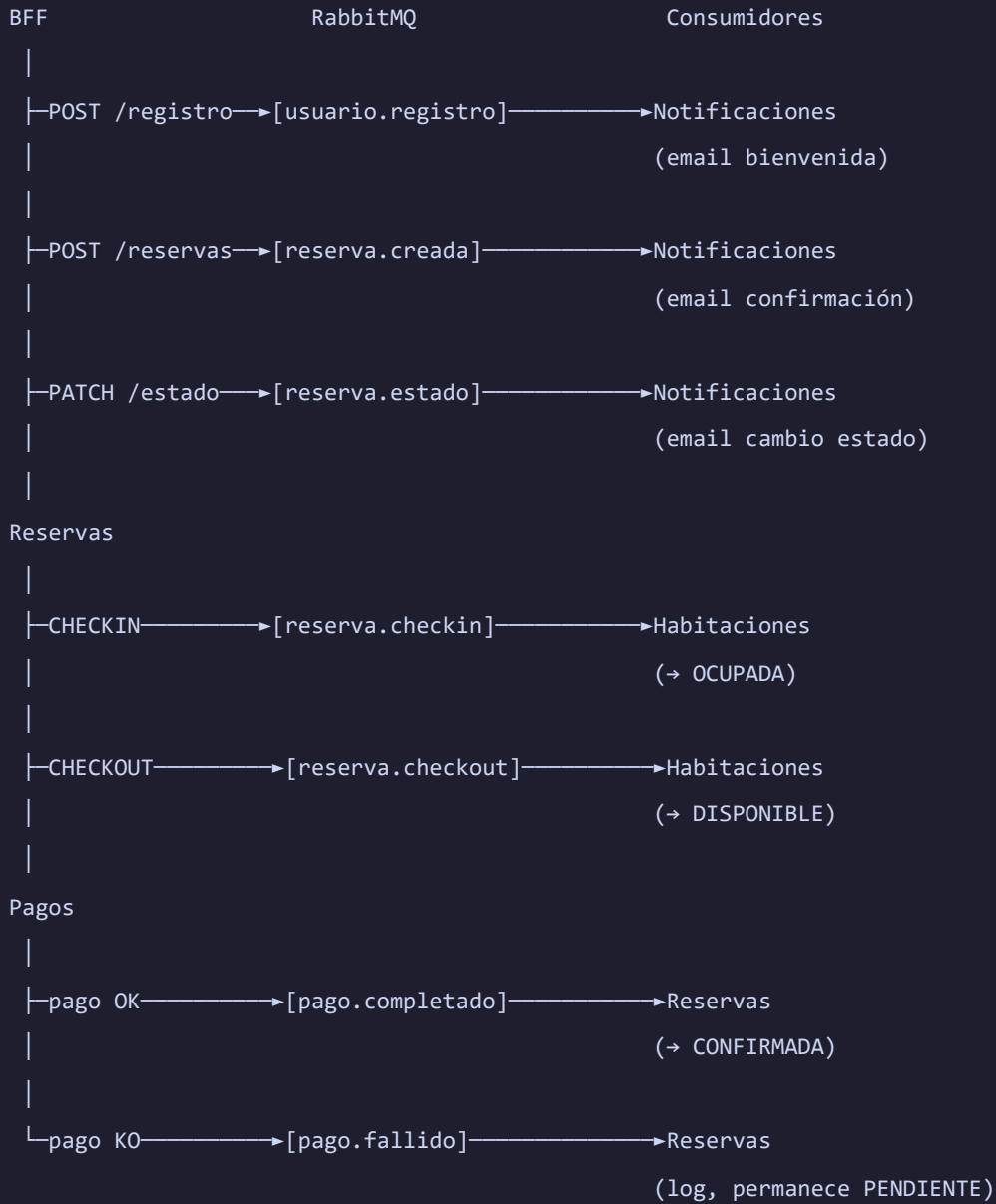
- |— habitacion.disponibilidad.actualizada → q.habitacion.estado (OTA)

Política de reintentos

```
spring.rabbitmq.listener.simple.retry:  
  enabled: true  
  max-attempts: 3  
  initial-interval: 1000ms    # 1 segundo  
  multiplier: 2               # Backoff exponencial  
  max-interval: 10000ms      # Máximo 10 segundos
```

Después de 3 intentos fallidos, el mensaje va al DLQ.

Diagrama completo de flujo de mensajes



15. Seguridad y Autenticación JWT

Flujo de autenticación

```
1. POST /bff/auth/login {email, password}
  → BFF → usuarios-service → verifica BCrypt → genera JWT + RefreshToken
  ← {accessToken, refreshToken, expiresIn}
```

1. Todas las llamadas subsecuentes:

Header: Authorization: Bearer <accessToken>

→ BFF valida JWT (firma + expiración + rol)

→ BFF reenvía Authorization al microservicio destino

```
1. POST /bff/auth/refresh {refreshToken}
```

→ Valida refresh token en BD → genera nuevo accessToken

← {accessToken, expiresIn}

```
1. POST /bff/auth/logout {refreshToken}
```

→ Elimina refresh token de BD

← 204 No Content

Estructura del JWT

```
Header: {"alg": "HS256", "typ": "JWT"}
```

Payload:

```
{
  "sub": "juan@hotel.com",      // email del usuario
  "rol": "HUESPED",             // rol para autorización
  "iat": 1717200000,            // issued at
  "exp": 1717286400             // expira (+ 24h)
}
```

```
Signature: HMAC-SHA256(base64Header + "." + base64Payload, JWT_SECRET)
```

Endpoints públicos (sin JWT)

```
POST /api/v1/auth/login
POST /api/v1/auth/registro
POST /api/v1/auth/refresh
GET /actuator/health
```

Roles y permisos

Operación	HUESPED	RECEPCIONISTA	ADMIN
Ver perfil propio	✓	✓	✓
Ver otras reservas	✗	✓	✓
Crear reserva	✓	✓	✓
Cambiar estado reserva	✗	✓	✓
Gestionar habitaciones	✗	PATCH estado	✓ CRUD
Gestionar usuarios	✗	Ver	✓ CRUD
Dashboard	✗	✓	✓

16. Resiliencia y Manejo de Fallos

Circuit Breaker (Resilience4j en BFF)

Configurado en todos los `*Client` del BFF:

```
resilience4j.circuitbreaker:
  instances:
    habitaciones-cb:
      slidingWindowSize: 10
      failureRateThreshold: 50      # % de fallos para abrir el circuito
      waitDurationInOpenState: 10s  # Tiempo en estado abierto
      slowCallDurationThreshold: 3s  # Llamada lenta
      permittedNumberOfCallsInHalfOpenState: 3
```

Cuando el circuito está abierto, se ejecuta el **fallback**:

```
// Ejemplo: si habitaciones-service no responde
public HabitacionResponse fallbackObtenerHabitacion(Long id, Throwable t) {
    return HabitacionResponse.builder().id(id).error("Servicio temporalmente no disponible").build();
}
```

Manejo de excepciones en microservicios

Cada servicio tiene un `@ControllerAdvice` que mapea excepciones a códigos HTTP:

Excepción	HTTP	Descripción
ReservaNotFoundException	404	Recurso no encontrado
HabitacionNoDisponibleException	409	Conflicto de fechas
CancelacionRestringidaException	422	Regla de negocio violada
TransicionEstadoInvalidaException	400	Estado inválido
PagoFallidoException	400	Error en pago
EmailDuplicadoException	409	Email ya registrado
MethodArgumentNotValidException	400	Bean Validation fallida
BadCredentialsException	401	Credenciales incorrectas

Formato de error estándar:

```
{
  "status": 409,
  "message": "La habitación 5 no está disponible para las fechas seleccionadas",
  "timestamp": "2026-12-01T15:30:00",
  "validationErrors": null
}
```

17. Flujos Completos del Sistema

Flujo 1: Registro de huésped

Frontend → POST /bff/auth/registro

```
{nombre: "Ana", email: "ana@mail.com", password: "Pass@123", rol: "HUESPED"}
```

BFF:

1. Llama POST /api/v1/auth/registro en usuarios-service

2. Recibe UsuarioResponse {id: 5, email: "ana@mail.com", ...}

3. Publica a RabbitMQ:

Exchange: staysync.notificaciones

Key: usuario.registro

Payload: {usuarioId: 5, nombre: "Ana", email: "ana@mail.com"}

Notificaciones (async):

4. Listener recibe UsuarioRegistradoEvent

5. Busca plantilla "USUARIO_REGISTRO"

6. Renderiza: "Bienvenida Ana a StaySync!"

7. Envía email via SMTP Brevo

8. Guarda Notificacion {estado: ENVIADO}

Frontend ← {accessToken, refreshToken, email, rol}

Flujo 2: Crear y pagar una reserva

Frontend → POST /bff/reservas

```
{habitacionId: 3, fechaEntrada: "2026-12-10", fechaSalida: "2026-12-15", numHuespedes: 2}
```

BFF → reservas-service POST /api/v1/reservas

reservas-service:

1. Verifica conflicto de fechas para habitación 3
2. Llama habitaciones-service → {precioPorNoche: 90.00, capacidad: 3}
3. Calcula precioTotal = 90 × 5 noches = 450
4. Guarda Reserva {estado: PENDIENTE, codigo: "RES-A1B2C3D4", precioTotal: 450}

BFF publica evento:

Exchange: staysync.notificaciones, Key: reserva.creada

→ Notificaciones envía email de confirmación

Frontend ← {id: 12, codigo: "RES-A1B2C3D4", estado: PENDIENTE, ...}

Frontend → POST /bff/pagos/stripe/checkout

```
{reservaId: 12, monto: 450.00, tituloReserva: "Habitación 301 - 5 noches"}
```

pagos-service:

1. Construye SessionCreateParams (monto, URLs de éxito/cancel)
2. Llama Stripe API → Stripe crea sesión de pago
3. Retorna {sessionId: "cs_test_ABC", checkoutUrl: "https://checkout.stripe.com/..."}

Frontend ← {checkoutUrl, sessionId}

Frontend redirige al usuario a checkoutUrl (paga con tarjeta en Stripe)

Stripe redirige → successUrl = /huesped/pago-exitoso?session_id=cs_test_ABC

Frontend → POST /bff/pagos/stripe/confirmar

```
{sessionId: "cs_test_ABC", usuarioId: 5}
```

pagos-service:

1. Verifica idempotencia: existsByGatewayId → false
2. Llama Stripe API: Session.retrieve("cs_test_ABC")
3. Verifica paymentStatus === "paid"
4. Registra Pago {estado: COMPLETADO, referencia: "STRIPE-XY1234", monto: 450, moneda: "CLP"}
5. Publica: Exchange staysync.pagos, Key: pago.completado
6. (Internamente) llama reservas-service PATCH /estado → CONFIRMADA

```
reservas-service recibe pago.completado:
```

```
→ Cambia Reserva a CONFIRMADA
```

```
→ Publica reserva.confirmada → Notificaciones (email adicional)
```

```
Frontend ← {id: 8, referencia: "STRIPE-XY1234", estado: COMPLETADO}
```

Flujo 3: Check-in

Recepcionista → PATCH /bff/reservas/12/estado {estado: "CHECKIN"}

BFF → reservas-service PATCH /api/v1/reservas/12/estado

reservas-service:

1. Verifica transición CONFIRMADA → CHECKIN (válida)
2. Cambia estado a CHECKIN
3. Publica: Exchange staysync.reservas, Key: reserva.checkin
Payload: {reservaId: 12, habitacionId: 3, ...}

habitaciones-service recibe reserva.checkin:

→ Cambia habitación 3 a estado OCUPADA

BFF publica evento:

Exchange: staysync.notificaciones, Key: reserva.estado
→ Notificaciones envía email "Tu check-in ha sido registrado"

Frontend ← {id: 12, estado: CHECKIN, ...}

18. Variables de Entorno

Variables requeridas (`.env` — nunca subir a git)

```
# MySQL
MYSQL_ROOT_PASSWORD=supersecreto
MYSQL_USER=staysync
MYSQL_PASSWORD=staysync_pass
```

JWT (mínimo 64 caracteres para HMAC-SHA256)

```
JWT_SECRET=tu-clave-secreta-muy-larga-aqui-minimo-64-caracteres-para-produccion
```

Stripe (NUNCA hardcodear en código)

```
STRIPE_SECRET_KEY=sk_test_...
```

RabbitMQ

```
RABBITMQ_USER=staysync
```

```
RABBITMQ_PASS=rabbit_pass
```

Email (Brevo / Mailtrap)

MAIL_HOST=smtp-relay.brevo.com

MAIL_PORT=587

MAIL_USERNAME=tu-email@brevo.com

MAIL_PASSWORD=tu-smtp-key

Para producción (ECR)

ECR_REGISTRY=123456789.dkr.ecr.us-east-1.amazonaws.com

IMAGE_TAG=1.0.0

Archivos de ejemplo

- `.env.example` — en la raíz del proyecto
 - `Frontend-StaySync-StaySync_Front_v1.1/.env.example` — variables de Vite (`VITE_*`)
-

19. Cómo Levantar el Proyecto

Prerequisitos

Herramienta	Versión mínima
Docker Desktop	24+
Java JDK	17
Maven	3.9+
Node.js	20+
npm	10+

Opción A: Docker Compose (todo el stack)

```
# 1. Clonar y entrar al directorio  
cd StaySync
```

2. Crear variables de entorno

```
cp .env.example .env
```

editar .env con valores reales

3. Levantar todo

```
docker compose up -d
```

4. Verificar que todos los servicios estén UP

```
docker compose ps
```

5. Verificar health de cada servicio

```
curl http://localhost:8080/actuator/health # BFF
```

```
curl http://localhost:8081/actuator/health # Usuarios
```

```
curl http://localhost:8082/actuator/health # Habitaciones
```

... etc

6. Abrir la app

```
open http://localhost:3000
```

Opción B: Servicios individuales (desarrollo)

```
# Infraestructura mínima (MySQL + RabbitMQ)
docker compose up -d mysql-usuarios rabbitmq
```

Usuarios-service

```
cd StaySync_Usuarios-usuarios_v1.0/StaySync_Usuarios-usuarios_v1.0
```

```
mvn spring-boot:run
```

En otra terminal – BFF

```
cd StaySync_BFF-bff_v1.0/StaySync_BFF-bff_v1.0
```

```
mvn spring-boot:run
```

Frontend

```
cd Frontend-StaySync-StaySync_Front_v1.1/Frontend-StaySync-StaySync_Front_v1.1
```

```
npm install
```

```
npm run dev
```

Ejecutar tests

```
# Tests de un microservicio
cd StaySync_Reservas-reservas_v1.1/StaySync_Reservas-reservas_v1.1
mvn test
```

Tests + reporte de cobertura JaCoCo

```
mvn verify
```

Abrir: <target/site/jacoco/index.html>

Solo unitarios (rápido)

```
mvn test -Dtest="ServiceTest,ListenerTest"
```

Solo integración

```
mvn test -Dtest="*IntegrationTest"
```

Frontend

```
cd Frontend-StaySync-StaySync_Front_v1.1/Frontend-StaySync-StaySync_Front_v1.1
```

```
npm run test
```

```
npm run coverage
```

RabbitMQ Management UI

URL: `http://localhost:15672`

Usuario: `staysync` (o valor de `RABBITMQ_USER`)

Contraseña: (valor de `RABBITMQ_PASS`)

Desde aquí puedes:

- Ver exchanges y bindings
- Monitorear colas (mensajes pendientes, DLQs)
- Publicar mensajes de prueba manualmente

Resumen rápido de la arquitectura

SERVICIO	PUERTO	DB	FUNCIÓN
Frontend	3000	–	React SPA
BFF	8080	–	Gateway/Auth
Usuarios	8081	usuarios_db	Auth + CRUD
Habitaciones	8082	habitaciones_db	Inventario
Reservas	8083	reservas_db	Ciclo reserva
Servicios	8084	servicios_db	Servicios hotel
Pagos	8085	pagos_db	Pagos + Stripe
Notificaciones	8086	notificaciones_db	Emails async
OTA	8087	ota_db	Canales extern.
RabbitMQ AMQP	5672	–	Message broker
RabbitMQ UI	15672	–	Monitoreo